# AIWEATHERWISE–DELIVER INTELLIGENT WEARTHER FORECASTS AND INSIGHTS

Documentation

## DESCRIPTION

AI WeatherWise is a robust RESTful backend platform built with Node.js and Express.js, utilizing MongoDB and MongoDM for data persistence. It delivers real–time weather metrics integrated with Google Gemini AI to provide intelligent summaries and personalized recommendations based on live environmental conditions.

The platform features secure JWT authentication, bcrypt password hashing, and a comprehensive suite of endpoints for managing favorite locations and fetching weather insights. It includes a resilient fallback mode ensuring API functionality even without external API keys, alongside centralized error handling and request sanitization.

Team Members Name & Roles:

1. Name:kavipriya D(Team Leader)

Role:Coding

2. Name:kavipriya L(Team Member)

Role:Testing video

3. Name: Jershitha B(Team Member)

Role:Demo voiceover

4. kaviyashree S(Team Member)

Role:Demo

5.Kalaivani R(Team Member)

Role:Documentation


Edit with WPS Office

# SCENARIO–BASED CASE STUDY

## Background

Alex is a travel enthusiast who frequently visits different cities and needs a streamlined way to track local weather conditions and receive AI–driven activity suggestions based on current forecasts.

## Problem

- Difficulty tracking and managing multiple favorite weather locations efficiently.
- Lack of personalized recommendations for outdoor activities or clothing.
- Need for natural language weather summaries rather than just raw data metrics.
- Absence of a secure system to store and sync preferences across devices.

## Solution

AI WeatherWise provides a centralized service that handles weather data retrieval and AI orchestration. It manages a secure User and Location database, allowing customized insights and automated recommendation workflows through Google Gemini.

## Usage

User Usage:

- Manages favorite cities (CRUD).
- Fetches real–time weather.
- Requests AI insights for recommendations and summaries.

Developer/Admin Usage:

- Monitors API health.
- Manages global user registrations.
- Verifies weather and AI service integrations.

Weather Application User:

- Interfaces with public weather endpoints to consume current metrics without requiring an account.

## Outcome

By leveraging AI and centralized data management, users spend less time


Edit with WPS Office

deciphering raw metrics. The system ensures data integrity through Mongoose schemas and provides high–value, context–aware weather advice through Gemini integrations.

## 1. SOFTWARE REQUIREMENTS

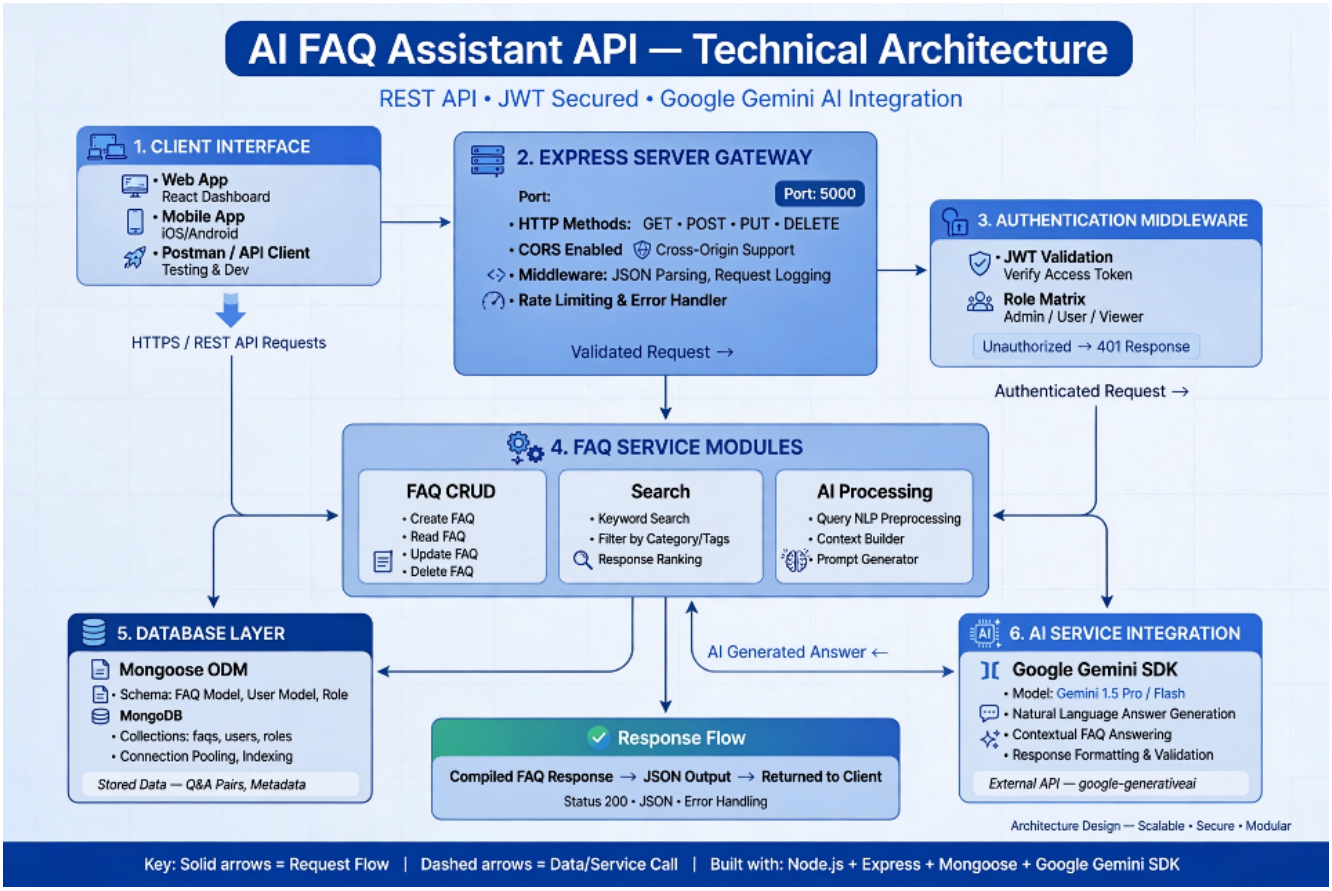
- Operating System: Windows 10/11, macOS, or Linux (supports cross–platform operations).
- Node.js (v16 or above): Runtime ecosystem running server logic and routing infrastructure.
- npm (v8 or above): Node package manager used to control operational dependencies.
- Gemini AI SDK: Integration using @google/generative–ai for natural–language insights.
- MongoDB: Document–oriented NoSQL database for storing users, locations, weather–related data, and analytics metrics.
- Postman: API verification toolkit used to test REST endpoints and validate responses.
- Code Editor: Visual Studio Code or a similar IDE environment.

## 2. HARDWARE REQUIREMENTS

- Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- RAM: 8 GB minimum; 16 GB recommended for concurrent MongoDB, processing layers, and execution nodes.
- Storage: 1 GB of available disk workspace.


Edit with WPS Office

## EPIC 1: PROJECT ARCHITECTURE




The AI WeatherWise application follows the MVC pattern to decouple business logic from data management. The Controller layer orchestrates weather fetching and generation of AI insights via Gemini, while the Model layer manages User and Location entities.

### Components Definition

### 1. Interface


Edit with WPS Office

API tools such as Postman query distinct route paths to extract JSON entities.

2. Express Server Gateway

Monitors network ports, maps HTTP methods (GET, POST, PUT, DELETE), parses JSON payloads, and handles CORS headers.

3. Authentication & Role Matrix Middleware

Extracts tokens from request headers, validates user–context signatures, and evaluates whether the user's role satisfies authorization requirements for protected routes.

4. Granular Services (Logic Modules)

Enforces isolated domain rules for application workflows, reporting indicators, and related service operations.

5. Database Interface Layer

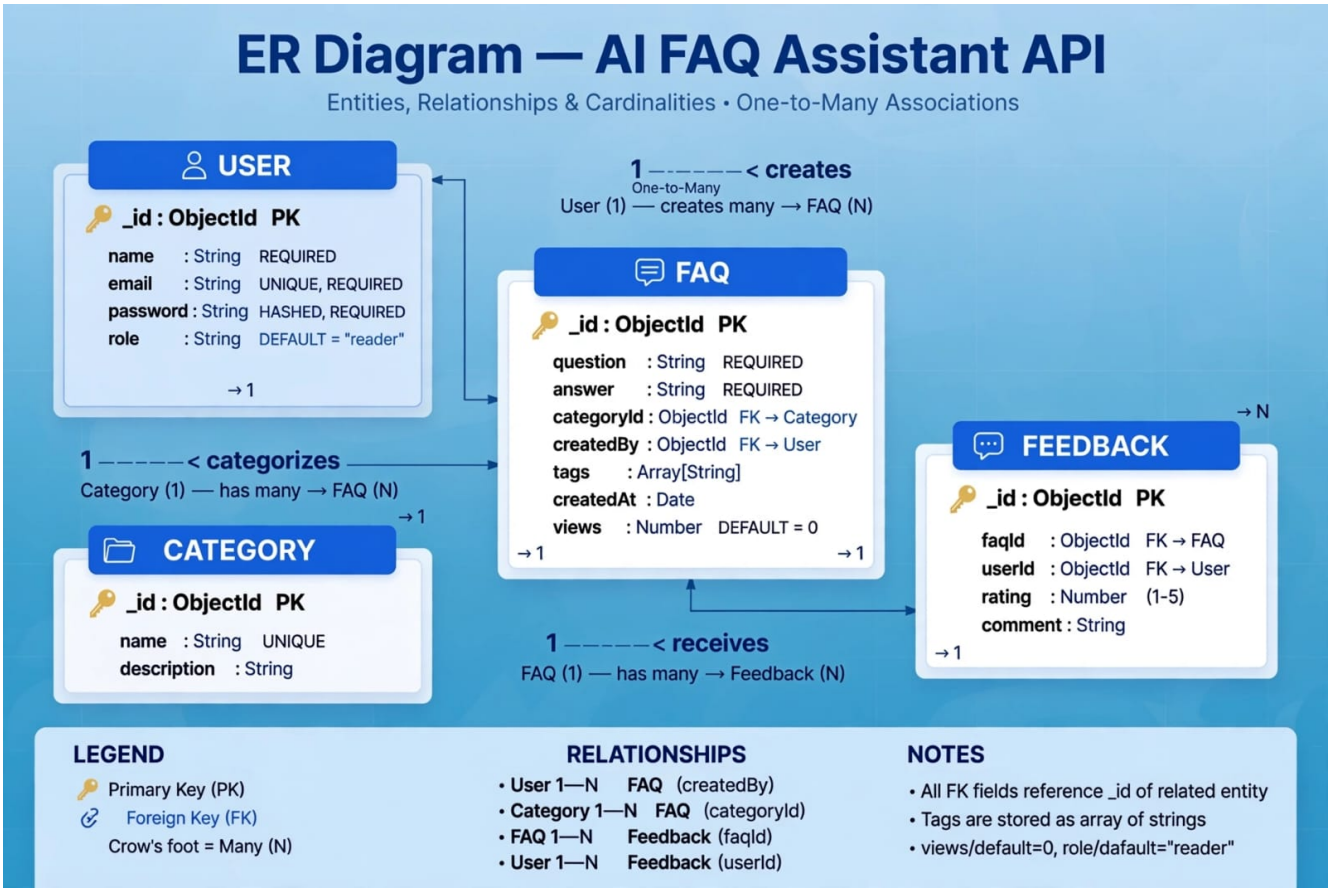
Mongoose translates application logic into MongoDB queries and manages schema validation and persistence.

6. AI Service Integration

Leverages the Google Gemini SDK to generate weather summaries and personalized recommendations based on live temperature and weather conditions.

## ENTITY–RELATIONSHIP (ER) DIAGRAM DESCRIPTION


Edit with WPS Office




The data architecture manages User profiles and their associated favorite Locations through logical references.

1. User Entity

- _id – Primary Key (ObjectId)
- name – String, Required
- email – String, Required, Unique


Edit with WPS Office

- password – String, Hashed, Required
- role – String, Default: "reader"

2. Location Entity

- _id – Primary Key (ObjectId)
- user – Foreign Key referencing User Collection
- city – String, Required
- country – String, Required
- createdAt – Date, Default: Now

Key Structural Mapping

User to Location (One–to–Many): A registered user can save multiple favorite locations.

## KEY FEATURES

- Role–Based Route Guarding: Strict token processing blocks unauthorized users from modifying protected resources.
- CRUD Location Management: Users can create, read, update, and delete favorite locations.
- Categorization and Querying: Native database indexing can support efficient filtering and discovery.
- Sanitization Safeguards: Payload sanitation helps prevent XSS vectors and NoSQL parameter–manipulation attacks.AI–Powered Recommendations: Gemini generates natural–language weather summaries and activity recommendations.

## ROLES AND RESPONSIBILITIES

### Admin

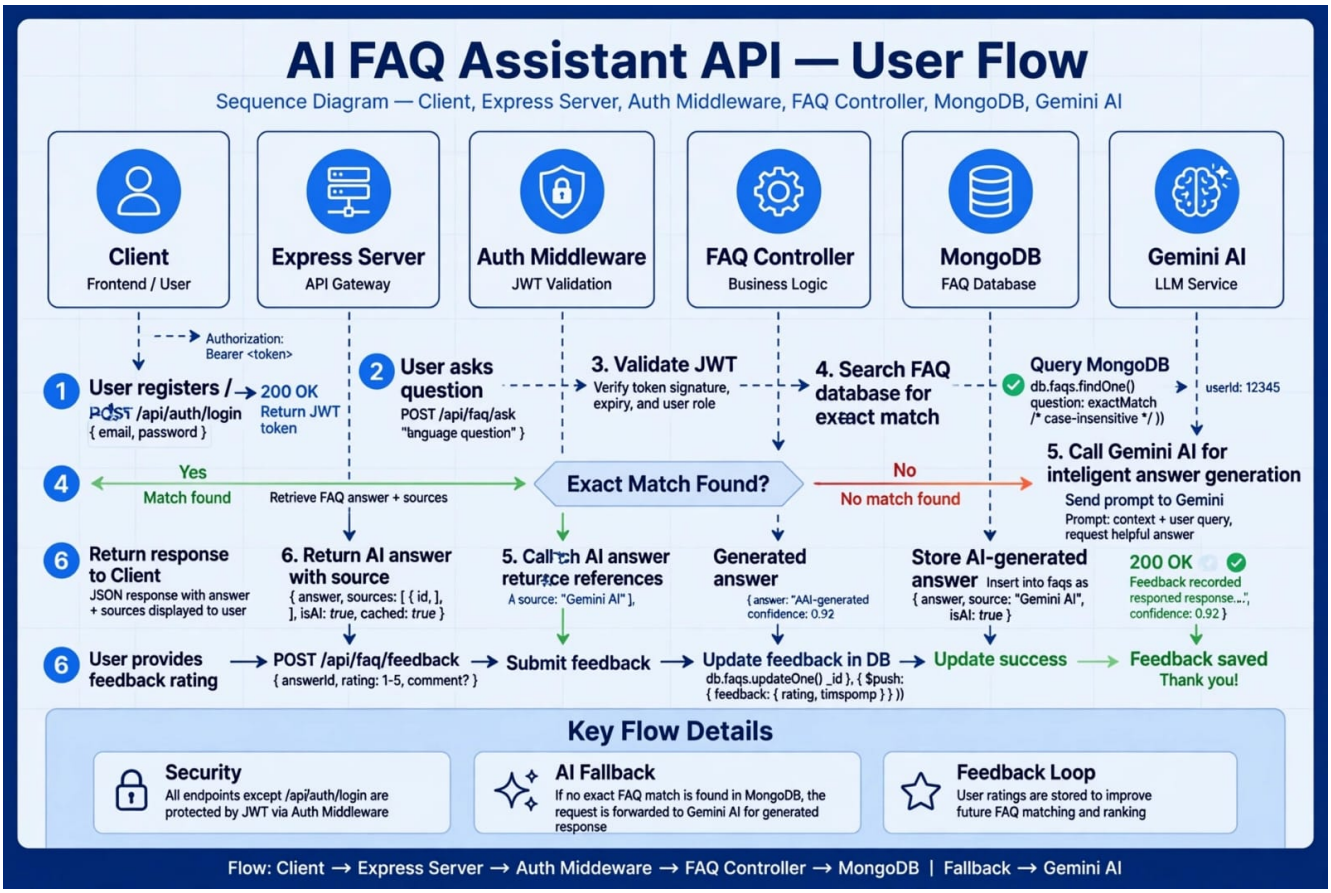
- Responsible for user account management, including creating, deleting, or suspending accounts.
- Oversees API health and system logs to monitor performance.
- Monitors and manages database integrity.


Edit with WPS Office

### Registered User

- Creates and maintains a list of favorite locations.
- Initiates requests for real–time weather data.
- Interacts with the Gemini AI service for personalized weather summaries and activity recommendations.
- Manages personal authentication credentials and profile details.

## USER FLOW





Edit with WPS Office

1. User logs in or registers.

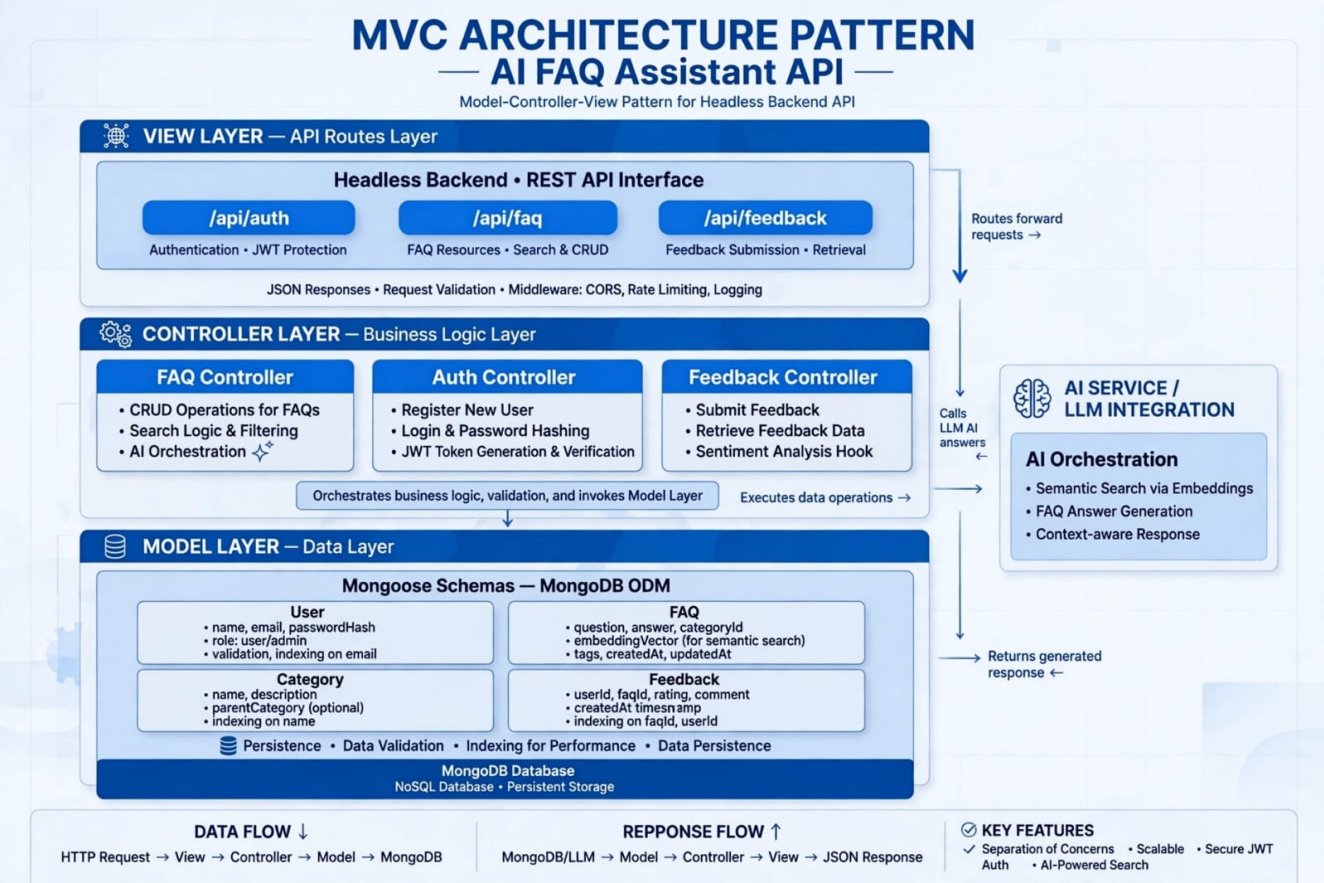
2. User saves a favorite city through the API.

3. User fetches current weather metrics for that city.

4. User requests AI–driven insights based on the weather data.

5. The system validates the JWT and returns the requested information.

## MVC ARCHITECTURE PATTERN





Edit with WPS Office

The system architecture structures its source configuration layers around the Model–View–Controller design pattern to decouple application components.

### Model Layer (Data Blueprint)

Defines structure blueprints using Mongoose schemas that map explicit fields to MongoDB collections. It sets default states, indexing rules, and validation parameters before data is persisted.

### Controller Layer (Execution Matrix)

Acts as the intermediary between routes and models. It captures requests, validates request constraints, delegates workflows to models and services, and packages results into standard JSON responses.

### View Layer (Routing Network Interface)

Because this is a headless backend service, traditional user–interface templates are not used. Instead, the interface layer is represented by the API routing layer, which links HTTP requests to controller handlers.

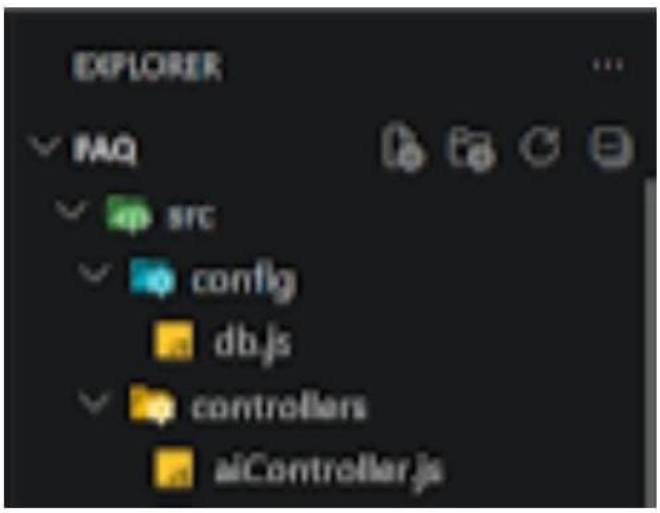
## EPIC 2: PROJECT SETUP AND CONFIGURATION

### Creating the Project Folder

1. Create a new folder named AI WeatherWise.

2. Open the project folder in Visual Studio Code.

3. Initialize the project using Node.js and install the required dependencies.





Edit with WPS Office

## Project Initialization

Open the project folder in the VS Code terminal and execute:

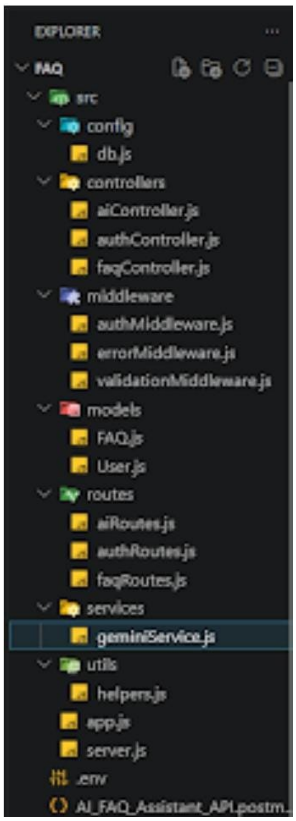
npm init –y

Install the required dependencies:

npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai

Install Nodemon for development:

npm install ––save–dev nodemon




## Create the Following Files

- index.js
- .env
- package.json


Edit with WPS Office

- .gitignore

### Create the Following Folder Structure

- controllers/
- models/
- routes/
- middleware/
- services/
- config/
- index.js
- .env
- package.json
- .gitignore

### Environment Configuration

Create a .env file in the project root and configure the required environment variables:

```
PORT=5000

MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_jwt_secret

GEMINI_API_KEY=your_gemini_api_key
```

## EPIC 3: BACKEND DEVELOPMENT

The AI WeatherWise backend is developed using Node.js, Express.js, and MongoDB following a modular architecture. The application separates routing, controllers, middleware, models, and utility modules to improve maintainability and scalability. The backend exposes secure REST APIs for authentication, weather information, favorite–location management, AI–powered insights, and administrative operations.


Edit with WPS Office



```
const app = require('./app');
const connectDB = async () => {
  try {
    const dbConnect = require('./config/db');
    await dbConnect();
  } catch (error) {
    console.error('Failed to connect to the database', error);
  }
};

const startServer = async () => {
  // Connect to database
  await connectDB();

  const PORT = process.env.PORT || 5000;

  const server = app.listen(PORT, () => {
    console.log(`Server running in ${process.env.NODE_ENV || 'development'} mode on port ${PORT}`);
  });

  // Handle unhandled promise rejections
  process.on("unhandledRejection", (err, promise) => {
    console.error(`Unhandled Rejection: ${err.message}`);
    // Close server & exit process
    server.close(() => process.exit(1));
  });
};

startServer();
```

## BACKEND STRUCTURE

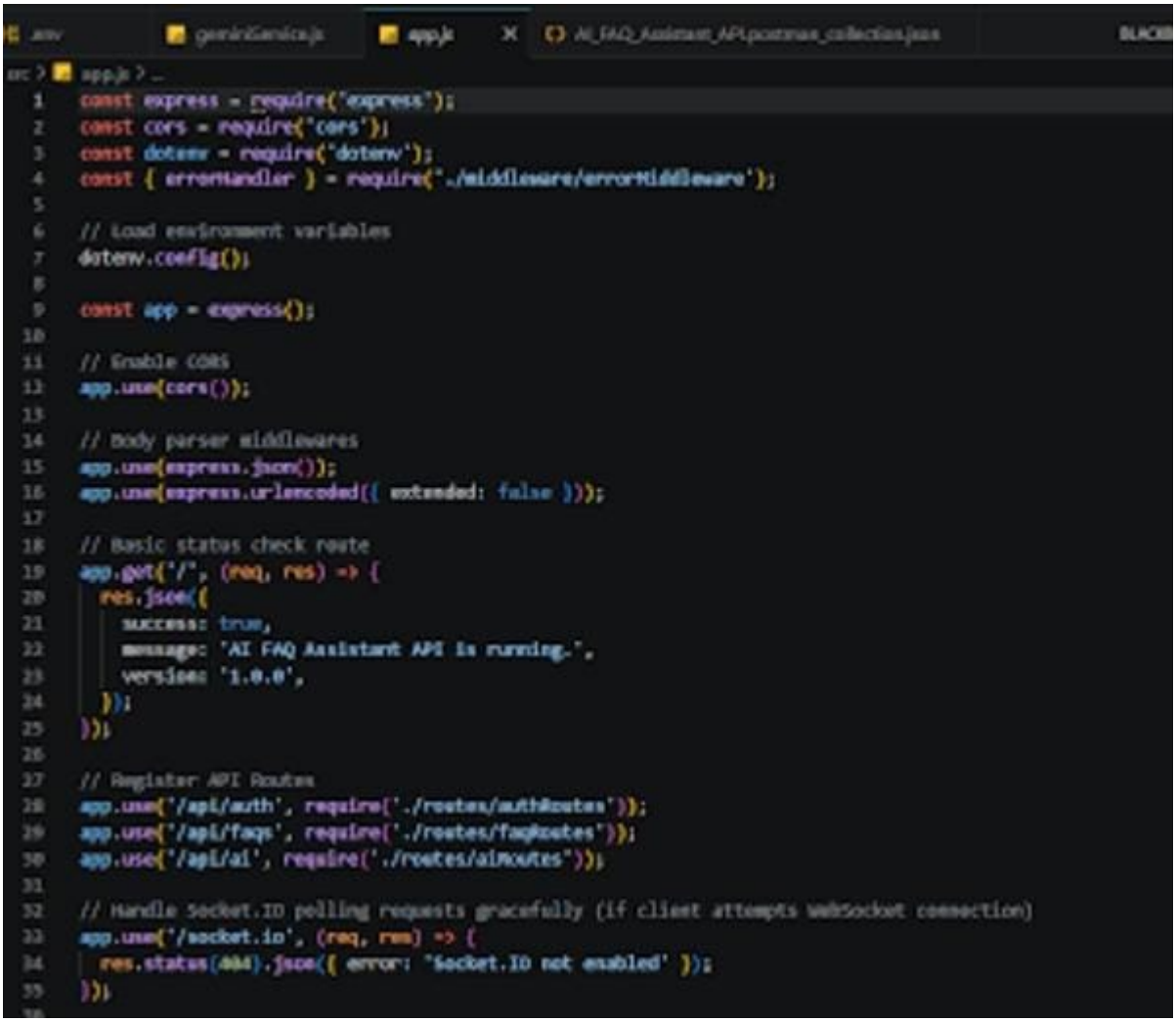
## Core System Script Implementations

### 1. Application Entry Point (index.js)

The index.js file serves as the main entry point of the application. It initializes the Express server, loads environment variables, configures middleware, establishes the MongoDB connection, registers API routes, and starts the server after a successful database connection.



Edit with WPS Office



```
.env    generalGeneric.js    app.js    AI_FAQ_Assistant_API.postman_collection.json    BLACKB
src > app.js > ...
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const { errorHandler } = require('./middleware/errorMiddleware');

// Load environment variables
dotenv.config();

const app = express();

// Enable CORS
app.use(cors());

// Body parser middlewares
app.use(express.json());
app.use(express.urlencoded({ extended: false }));

// Basic status check route
app.get('/', (req, res) => {
  res.json({
    success: true,
    message: 'AI FAQ Assistant API is running.',
    version: '1.0.0',
  });
});

// Register API Routes
app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/faqs', require('./routes/faqRoutes'));
app.use('/api/ai', require('./routes/aiRoutes'));

// Handle Socket.IO polling requests gracefully (if client attempts WebSocket connection)
app.use('/socket.io', (req, res) => {
  res.status(404).json({ error: 'Socket.IO not enabled' });
});
```



Edit with WPS Office

# EPIC 4: DATABASE CONFIGURATION

## API Testing Demo

## 1. Configure MongoDB

Install Mongoose. In the Node.js project directory, run:

```
npm install mongoose
```

### Create Database Connection

Make sure the database is connected before performing backend operations.

## Create Schemas & Models

Define the User and Location schemas using Mongoose.

## Location Schema

- user – ObjectId reference to User
- city – String, required
- country – String, required
- createdAt – Date with default current time


Edit with WPS Office

```
const mongoose = require('mongoose');

const locationSchema = new mongoose.Schema({
  city: {
    type: String,
    required: [true, 'Please provide a city name'],
    trim: true
  },
  country: {
    type: String,
    required: [true, 'Please provide a country name'],
    trim: true
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: [true, 'User reference is required']
  }
}, {
  timestamps: true
});

// Ensure a user cannot duplicate the exact same city and country in their favorites
locationSchema.index({ user: 1, city: 1, country: 1 }, { unique: true });

const Location = mongoose.model('Location', locationSchema);
module.exports = Location;
```

## User Schema

- name – String, required
- email – String, required and unique
- password – String, required and stored as a hash
- role – String with default value "reader"


Edit with WPS Office

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please provide a name'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Please provide an email'],
    unique: true,
    trim: true,
    lowercase: true,
    match: [
      /^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
      'Please provide a valid email address'
    ]
  },
  password: {
    type: String,
    required: [true, 'Please provide a password'],
    minlength: [6, 'Password must be at least 6 characters']
  }
}, {
  timestamps: true
});
```

## Database Connection

This module establishes a connection between the application and MongoDB using Mongoose. It ensures that the backend communicates with the database before processing client requests.


Edit with WPS Office

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/weatherwise');
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

## EPIC 5: PROJECT EXECUTION

The AI WeatherWise backend can be executed locally using Node.js. Before running the application, ensure that MongoDB is running and the required environment variables are configured correctly. The backend exposes REST APIs that can be tested using Postman or Thunder Client.

### Running the Backend Server

1. Install all required dependencies:

    npm install

2. Start the development server:

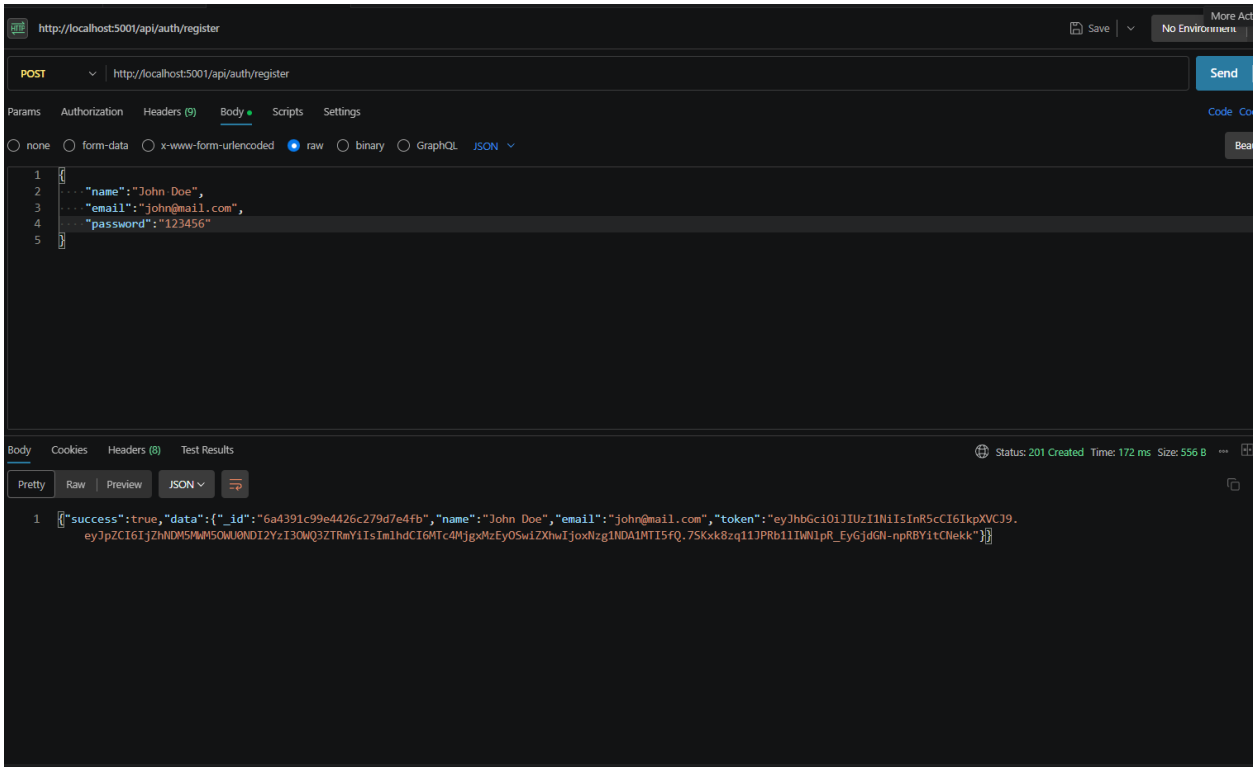
    npm start

After successful execution, the server connects to MongoDB and starts listening for incoming API requests.


Edit with WPS Office

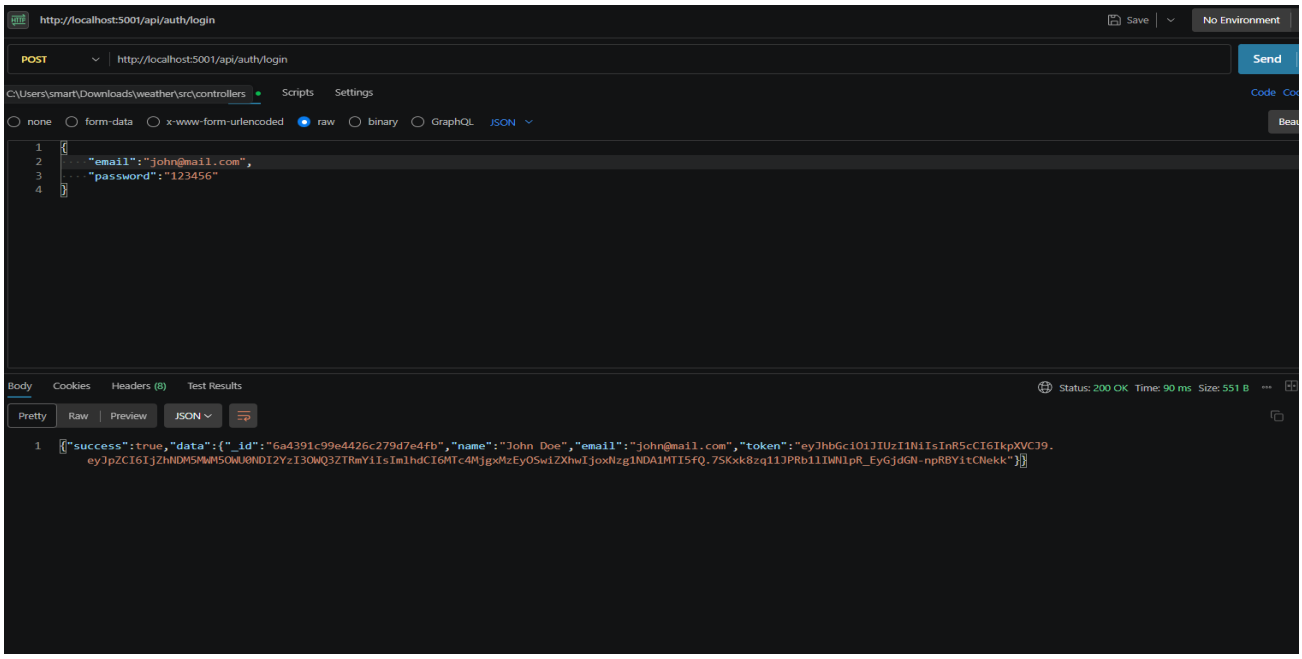
# API TESTING

## Register API




Used to create a new user account and securely store authentication credentials.

## Login API

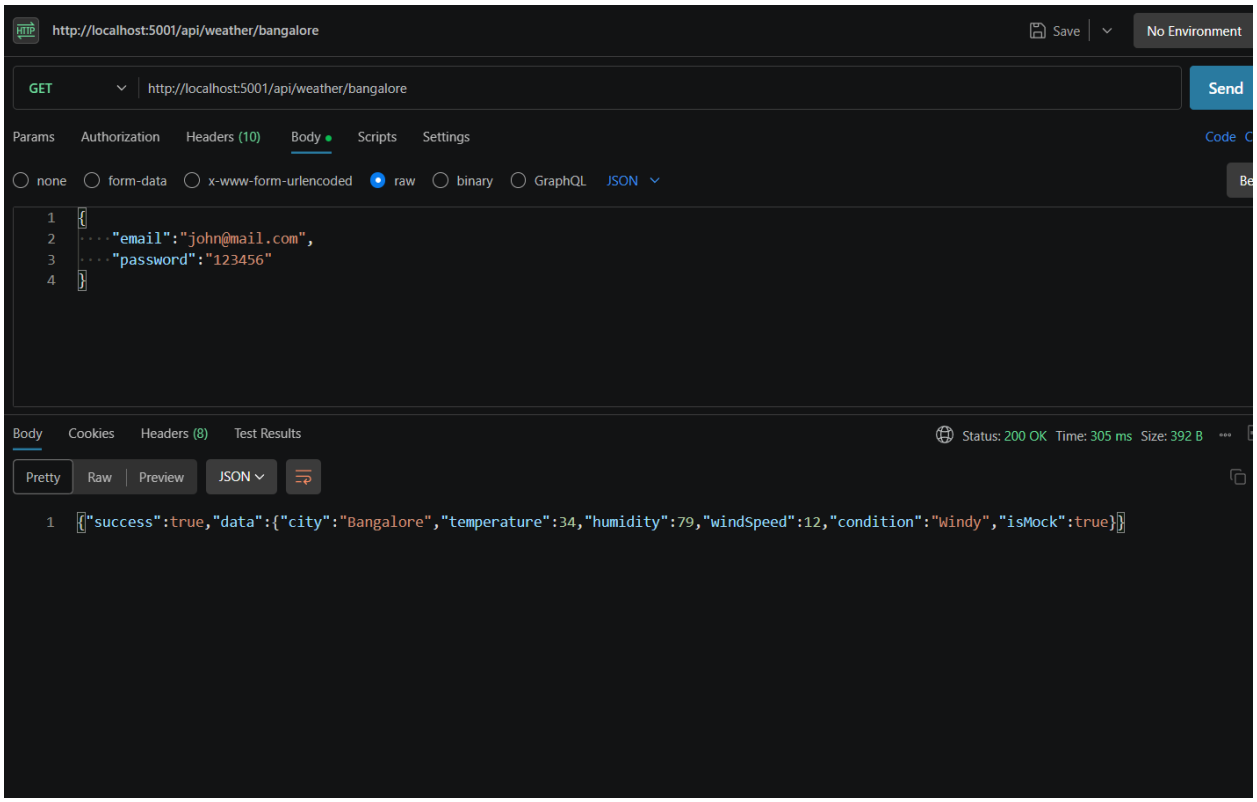




Edit with WPS Office

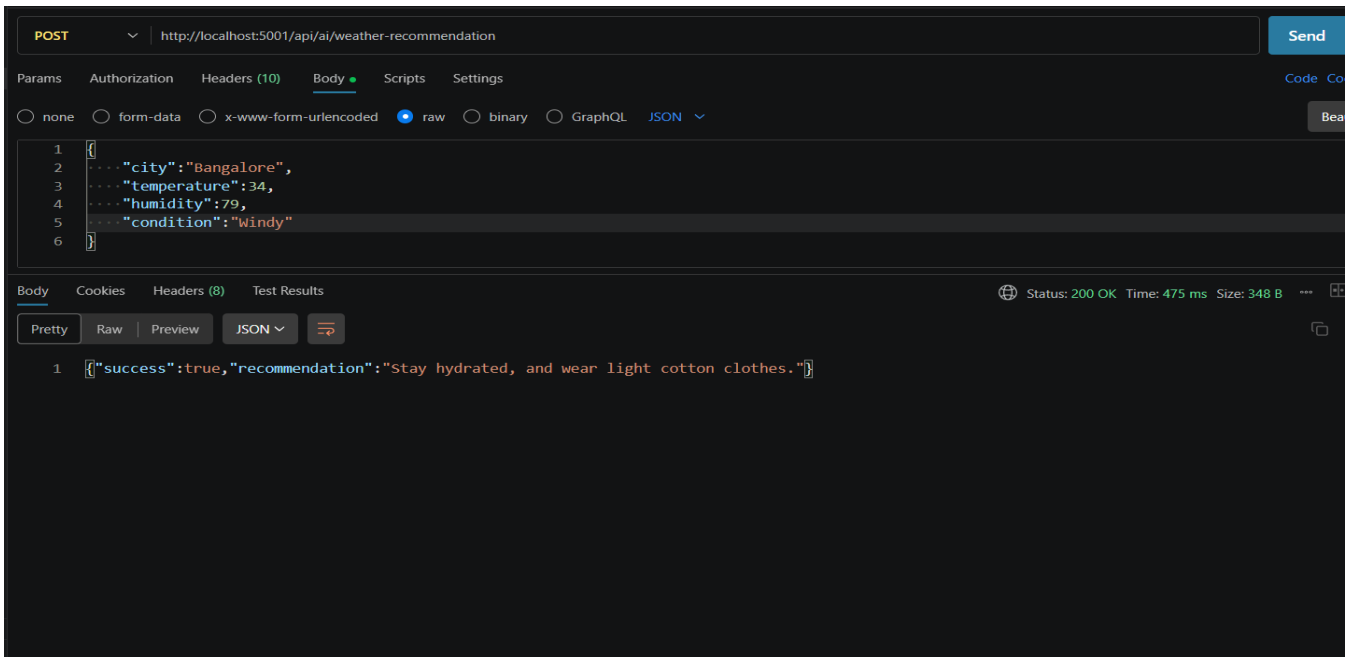
Used to authenticate a registered user and generate a JWT token.

## Weather API




Used to retrieve current weather metrics for a selected city.

## Recommendation API





Edit with WPS Office

Used to generate AI–powered weather summaries, activity suggestions, and personalized recommendations.

## CONCLUSION

AI WeatherWise combines real–time weather data, secure backend services, MongoDB persistence, and Google Gemini AI to provide intelligent and personalized weather insights. Its modular MVC architecture improves maintainability, while JWT authentication and password hashing strengthen security. The platform can be extended with additional weather providers, richer personalization, notification services, and mobile or web interfaces.


Edit with WPS Office